NexClean — Pre-Production Test & Bug Report

9 August 2026 · branch `main` @ `20fac9e` · backend + frontend + full E2E

All reported issues are fixed and verified.

Full suite: **385 passed, 0 failed, 0 flaky** from a clean database. Every finding was reproduced against a running server first, then re-tested after the fix — `tests/verify-fixes.mjs` replays that proof in one command (**10/10 passing**).

Four items were found and deliberately **not** changed, and several areas were never exercised — both are listed honestly in **§9**. Read that section before treating this as “done”.

At a glance

ID	ISSUE	IMPACT	STATUS
P0-0	Nobody could book at all in the production config	Revenue = zero	FIXED
P0-1	Direct booking route would bypass online checkout	Free bookings	FIXED
P0-2	One payment replayable into unlimited bookings	Revenue loss	FIXED
P0-3	Price tampering between order and verify	Revenue loss	FIXED
P0-4	`DEMO_OTP` logged in as *any* phone number	Account takeover	FIXED
P0-5	Any user could read/write/close any support chat	Data breach	FIXED
P0-6	“Refund” never returned any money	Consumer harm	FIXED
P0-7	New cleaners could never finish onboarding	No supply growth	FIXED
P0-8	Production booted on development secrets	Forgeable tokens	FIXED
P1-1	No rate limiting anywhere (OTP, SMS, admin login)	Brute force / cost	FIXED
P1-2	Suspending a user didn’t log them out	30-day access	FIXED
P1-3	No-coordinate bookings sent to arbitrary cleaners	Mis-routed jobs	FIXED
P1-4	Stack traces in API responses; 500s where 4xx belonged	Info disclosure	FIXED
P1-5	Before/after job photos were never actually captured	No dispute proof	FIXED
P1-6	Agent could inflate their own completed-job count	Fake stats	FIXED
P1-7	Bookings accepted with dates in the past	Bad schedule data	FIXED
P1-8	Real-time silently broken in the PM2 cluster (`REDIS_URL` unset)	Lost chat & job offers	CONFIG
P1-9	Landing page dragged sideways 19px on a phone	Mobile UX	FIXED
P1-10	Per-account cap overrun by parallel writes	Data growth	FIXED
P0-9	Razorpay account refuses every refund (not a code fault)	No refunds possible	ACCOUNT
P2	Fail-open admin perms · non-Latin names rejected · unbounded lists · unvalidated booking slots · unchecked image uploads	Hardening	FIXED
P1-11	No session revocation — a stolen token was valid for 30 days	Account takeover	FIXED
P1-12	Any cleaner could push a notification to any society	Push spam	FIXED
P1-13	Referral bonuses farmable with multiple phone numbers	Coin fraud	FIXED

P1-14	CSV export executable in Excel (public, unauthenticated input)	Admin compromise	FIXED
P1-15	`trust proxy` unset behind nginx — IP always nginx's own	Rate limits broken	FIXED
P1-16	Admin dashboard loaded every booking into memory	Timeout at scale	FIXED
P1-17	Session token readable by injected script (localStorage)	Token theft	FIXED

The most serious problem was not in the first report

It surfaced while fixing P0-1: **with the production `.env` as written, no customer could book at all.** The app moved to pay-to-cleaner (cash on completion) and the frontend stopped sending Razorpay fields, but the backend still switched itself into online-payment mode purely because Razorpay keys were present — so every booking returned `400 Missing payment confirmation fields`. Collection mode is now an explicit `PAYMENTS_ONLINE` switch instead of an inference.

A correction to the original report. I first flagged `POST /api/bookings` as a "free booking" vulnerability. That was wrong — under pay-to-cleaner, creating a booking without an online charge *is* the product. It only becomes a bypass once online checkout is enabled, so it is now closed off in exactly that mode rather than removed.

The foundations were sound throughout: pricing is server-authoritative, coin redemption is atomic, role separation holds, Zod blocks NoSQL injection, and no secret has ever been committed.

1 · What was tested, and how

LAYER	METHOD	RESULT
Backend typecheck	`tsc --noEmit`	Clean
Frontend build	`npm run build` (2,585 modules)	Clean
API suite	237 Playwright tests — every backend module, all 156 routes	87 failing → **all green**
E2E suite	~159 browser tests (landing / customer / agent / admin)	32 failing → **all green**, +14 new landing tests
Abuse & edge cases	25 adversarial + 7 concurrency + 3 multi-worker + 4 scheduler probes	11 defects + 3 minor → **all fixed**
Fix verification	`verify-fixes.mjs` + `live-integrations.mjs` + `volume.mjs`	**all passing**
Secret hygiene	`git ls-files` + full history scan	No `.env` ever committed

Everything ran against a disposable local MongoDB, never the production Atlas cluster.

Watch out when you run this yourself. `tests/run-all.sh` adopts whatever already listens on port 5173. On this machine that was a *different project* (`Ezoflife123-main`), so the default invocation would have silently tested the wrong app. Every run here used `FRONTEND_PORT=5199`. Worth making the script verify the app it adopts.

```
cd tests && FRONTEND_PORT=5199 ./run-all.sh   # full functional suite
node verify-fixes.mjs                          # replays the proof for every fix
```

2 · The two bugs that were blocking testing itself

F1 — Signup returned HTTP 500 for every user after the first

`referralCode` was declared `{ default: null, unique: true, sparse: true }`. A sparse index skips documents where the field is **missing** — but `default: null` writes an explicit `null` on every insert, so the *second* user without a referral code collided.

```
E11000 duplicate key error collection: nexclean-test.users
index: referralCode_1 dup key: { referralCode: null }
```

Every agent, and every customer created through OTP-only login, hit this. `config/db.ts` already tried to drop the index on boot, but Mongoose's `autoIndex` immediately rebuilt it from the schema, so the drop never stuck.

> **Fixed:** replaced with a partial index scoped to real strings, so nulls are ignored entirely. The existing `dropIndex` now migrates deployments on next boot. **This one change took the API suite from 87 failures to 16.**

F2 — The demo-agent seed was empty, silently killing all agent test coverage

`const AGENTS: AgentSeed[] = []` left 16 tests failing against agents that no longer existed — and, more importantly, meant **no agent functionality was being tested at all**.

> **Fixed:** restored the four demo cleaners, gated to `NODE_ENV === "test"` only. The gate is deliberately strict — this project's own `backend/.env` runs `NODE_ENV=development` against the **production** Atlas cluster, so anything looser would inject fake cleaners into live data.

3 · P0 — Blocking (all fixed)

P0-0 Nobody could book at all in the production configuration

The app collects **cash on completion**: the checkout button reads *"Pay to cleaner ₹…"* and `BookingFlow.tsx` sends the booking with **no Razorpay fields**. But the server decided it was in online-payment mode from nothing more than `RAZORPAY_KEY_ID` / `SECRET` being non-empty — and `backend/.env` sets both. In that mode `verifyAndBook()` rejects anything lacking a signature.

> **Proof:** a backend started with Razorpay keys, sent exactly the payload the current frontend sends → `400 Missing payment confirmation fields` . Booking was completely dead.

> **Fixed:** collection mode is now an explicit `PAYMENTS_ONLINE` flag (default `false` , documented in `.env.example`). Presence of keys alone no longer changes behaviour.

P0-1 `POST /api/bookings` would become a free-booking bypass

Under pay-to-cleaner there is no prepayment to bypass, so an unpaid booking is the intended flow. It *does* become a bypass the moment online checkout is switched on, since the route takes no order id and no signature.

> **Fixed:** the route is rejected when online payments are live, so enabling them can't silently reopen a free path. Verified: `400 Bookings must be created through the payment flow` .

P0-2 One payment → unlimited bookings (replay)

`verifyAndBook()` never marked an order consumed, so replaying the same order id + signature minted a new booking every time — one order redeemed 3× in testing. The Razorpay signature covers only `order_id|payment_id` , so it stays valid on every replay; this was exploitable in live mode.

> **Fixed:** the order is claimed with a conditional update, so the first verify wins and a replay finds nothing to claim. Verified with a genuine HMAC signature: first → `201` , identical replay → `400 This payment has already been used…`

P0-3 Price tampering between `/order` and `/verify`

The booking payload was sent twice and the two were never compared — so ordering a ₹300 Express Wash and verifying with an Ultimate Detail produced a ₹690 booking for ₹300.

> **Fixed:** the priced payload is stored on the Payment when the order is created, and the booking is built from **that**, never from the payload accompanying the signature. Verified: ordered "Express Wash", verified with "Ultimate Detail", received **"Express Wash" at ₹300**.

P0-4 `DEMO_OTP` was a universal master password

OTP verification accepted the demo code for *any* phone number with no environment gate, and `backend/.env` ships `DEMO_OTP=123456` — enough to log in as any customer or cleaner, including a specific known one, since accounts are keyed by phone number.

> **Fixed:** honoured only outside production, and the server now refuses to boot production with `DEMO_OTP` set at all. Verified in production mode: `123456` → `400 Request a new code` .

P0-5 Any user could read, write and close any other user's support chat

All four ticket operations took a bare `ticketId` with no ownership check.

> **Proof:** a second customer read the victim's message *"My card ending 4242 was double charged, ref #99812"*, posted into the thread, and closed it — all `200` . Ticket ids are enumerable ObjectIds.

> **Fixed:** one helper gates all four — staff see everything, everyone else only their own. It returns **404, not 403**, so ticket ids stay unprobeable. Verified: attacker read/write/close → 404 / 404 / 404.

P0-6 "Refund" never returned any money

The refund handler only flipped database fields — there was no Razorpay refund call anywhere in the codebase, while the app's own published policy promises money back in 5–7 business days. The amount was also unbounded, so a refund could exceed the original payment.

> **Fixed:** the gateway refund is called and the record is marked refunded **only after the gateway confirms**; a failure surfaces to the admin instead of silently "succeeding". Amount is clamped to the original payment.

P0-9 Refunds are refused by the Razorpay account itself

(Found by testing against the real provider — not visible from the code.)

With P0-6 fixed, the app now genuinely calls Razorpay. Driving that through the admin endpoint against the real (test-mode) account, **every refund is declined** — ₹1, ₹10 and a full refund all come back `400 "invalid request sent"` , on both card and UPI payments. Order creation and reads on the very same credentials work fine, so this is an **account-level restriction** — typically refunds not being enabled until activation/KYC completes — rather than a code fault.

> **Verified good news:** when the gateway refuses, the record correctly stays `status: "paid"` , `refundAmount: ₹0` — **no false "refunded"**. That is precisely the guarantee P0-6 was written for, now confirmed against the live API rather than assumed.

You do NOT need the Payouts API. Payouts (RazorpayX) send money to arbitrary bank accounts — a separate product, and indeed not enabled on this key. Refunds return money against a specific payment and live on the Payments API, which your key already has. Probing the key directly:

```
Payments  (needed for refunds)     HTTP 200  available
Orders    (checkout)               HTTP 200  available
Refunds   (list)                   HTTP 200  available
Payouts   (RazorpayX — separate)   HTTP 400  not enabled  ← not required
```

So the API is available and *reading* refunds works; only *creating* one is refused. The most likely reason is that there is nothing to fund it from — **₹2,979 of the ₹3,050 captured has already been settled out**, and Razorpay draws refunds from your current balance.

> **Do this next:** open the Razorpay Dashboard in test mode, find that payment and press Refund. The dashboard states the actual reason, where the API only returns a generic "invalid request sent". If it also refuses, Razorpay support can enable it or explain the balance requirement. Re-check afterwards with `node tests/live-integrations.mjs` .

P0-7 New cleaners could never finish onboarding

Signup creates the account as `agentStatus: "unverified"`, and the KYC endpoint accepts *only* that state — but the frontend's status union (`"verified" | "pending" | "suspended"`) **omitted "unverified"**. So the signup page bounced new cleaners out before they could upload documents, and every status check fell through. A dead end: they sat on a dashboard, couldn't reach KYC, and could never be verified — while dispatch and job-claiming both require "verified". **The entire cleaner acquisition funnel was broken.**

> **Fixed:** "unverified" added to the union, the signup page keeps those agents on the KYC step, and the restricted-dashboard checks now test `!== "verified"` so the existing "Submit KYC" banner actually appears. The full signup → OTP → KYC → pending-review path now passes end to end.

P0-8 Production configuration was not production-safe

Config silently fell back to `jwtSecret: "dev-secret-change-me"` and `adminPassword: "password123"`, so a missing env var in production yields **forgeable JWTs** rather than a boot failure.

> **Fixed:** a startup guard runs before the port is bound. Verified against the real `.env` values:

```
Refusing to start in production with an unsafe configuration:
  - ADMIN_PASSWORD is unset or still the development default
  - DEMO_OTP is set — it accepts any phone number and must be empty in production
  - RAZORPAY_KEY_ID is a TEST key — live payments would never be captured
```

4 · P1 — High (fixed, except one config change)

P1-1 No rate limiting anywhere

Confirmed by probe: 60 wrong OTP codes with no lockout (the `attempts` field existed but nothing ever incremented it), 30 OTP sends for one number (each a billable SMS), and 30 failed admin logins — all unthrottled.

> **Fixed:** `express-rate-limit` on the auth surface — OTP sends 5/15 min, verifies 10/15 min, admin login 10/15 min, area alerts 6/hour. Keyed on the **phone number** where present, so rotating IPs doesn't help. OTP codes are also burned after 5 wrong guesses. Verified: `429 after 10 failed attempts`.

Caveat: the store is in-memory, so under PM2 cluster mode limits are per-worker (≈ limit × workers). Still a hard ceiling — move to Redis if that's too loose; `REDIS_URL` is already configured.

P1-2 Suspending a user didn't log them out

Suspension was enforced only at OTP login, so a 30-day token kept working — a cleaner removed for misconduct kept full access for up to a month.

> **Fixed:** auth middleware now checks account status and rejects suspended users. Verified: before → `200`, after suspension → `403 ACCOUNT_DEACTIVATED`. Costs one indexed lookup per request, which is the price of real revocation.

P1-3 Bookings without coordinates were dispatched to arbitrary cleaners

The no-geo fallback offered the job to whichever verified agent had been seen most recently — possibly in another city — and its second pass dropped the heartbeat filter entirely. **Three independent tests failed on this**, which is what distinguished it from a stale expectation: the booking spec (`offered`, not `pending_admin`), the agent spec (a cleaner holding an offer with the tell-tale `"distanceKm": null`), and the admin dispatch spec (the job never reaching *"Needs assignment"*).

> **Fixed:** the fallback is removed — a booking with no service location goes straight to the admin queue, which is the documented behaviour. Geocoding failing is exactly when a human should place the job.

P1-4 Stack traces in API responses, and 500s where 4xx belonged

Every unrecognised error became a 500, and the stack was included whenever `NODE_ENV !== "production"` — which is exactly how `.env` is configured. A malformed ObjectId returned **500 with a full stack trace**.

> **Fixed:** known errors are mapped properly (bad id → 400, validation → 400, duplicate key → 409, bad JSON → 400, oversized body → 413), and stacks sit behind an explicit debug flag that is ignored in production. Verified: `400`, `stackLeaked=false`.

P1-5 Before/after job photos were never actually captured

The photo endpoint took only `{ kind }` — no image. It flipped a boolean, the Booking model had no photo field, and the agent UI picked a file then discarded it. The "photos required before completing" gate was satisfiable with two empty POSTs, and nothing was retained to settle a damage or no-show dispute.

> **Fixed:** the endpoint accepts an image, uploads it through the existing Cloudinary helper, and stores the URLs on the booking. The agent app now sends the file it already asks for. The image stays optional so older clients keep working.

P1-6 An agent could inflate their own completed-job count

The status clamp meant calling "advance" again on a completed job re-entered the completion branch and incremented `jobsDone` every time. `jobsDone` drives the cleaner's public stats.

> **Fixed:** advancing an already-terminal job returns `400 This job is already completed`.

P1-8 Real-time features are silently broken in the clustered deployment

`deploy.sh` restarts a **PM2 cluster** ("all cluster workers"), and the code has proper Redis-adapter support for exactly that — but `REDIS_URL` is **empty in `backend/.env`**. Socket.IO only knows about sockets in its own process, so any event emitted by the worker handling an HTTP request never reaches a customer or cleaner whose WebSocket is held by a different worker. With N workers, roughly (N–1)/N of real-time events are lost.

> **Proof:** two workers were run against one database. *With* `REDIS_URL` set, an event emitted on worker B arrived at a socket held by worker A. With it empty, the same test failed while the same-worker control still passed — which is precisely the production configuration.

What actually degrades: **live chat has no polling fallback at all** (customer, cleaner and admin panels are socket-only), and targeted "nearest cleaner first" dispatch offers arrive over `job_offer` — a cleaner on the wrong worker simply never sees the popup, and only picks the job up later from the 30-second available-jobs poll, if at all.

> **Action:** this one is **config, not code** — set `REDIS_URL` in the production environment. Redis is already a dependency and the adapter wiring is already correct; nothing needs changing in the source. Verify afterwards with `tests/multiworker.mjs`.

P1-9 The landing page could be dragged sideways on a phone

At 390 px the page scrolled **19 px horizontally**. Reveal animations park their elements at `translateX(40px)` until scrolled into view, and that offset widened the document. `body` already had `overflow-x: hidden`, but with `html` still `visible` it did not contain the viewport — the page really could be dragged, confirmed by scripting a horizontal scroll and reading back `window.scrollX = 19`.

> **Fixed:** `overflow-x: clip` on `html` and `body`. `clip` rather than `hidden` is deliberate — it doesn't create a scroll container, so the sticky headers in the customer, cleaner and admin layouts keep working (verified after the change). Now 0 px of horizontal scroll at 360, 390, 768 and 1280 px.

P1-10 The per-account cap could be overrun by parallel writes

Self-inflicted, and caught by the new concurrency probes: the vehicle/address cap added during this review was a read-then-write. Twenty simultaneous adds each read a length under the limit, and **17 landed against a cap of 15**.

> **Fixed:** replaced with a single atomic update whose filter (`{ "vehicles.14": { $exists: false } }`) asserts "fewer than 15 entries" on the server as part of the same write. Stable at exactly 15 across repeated 20-way bursts.

P1-7 Bookings accepted with dates in the past

The date was validated only as an ISO datetime; a booking dated 30 days ago was accepted and entered dispatch.

> **Fixed:** bounded from 24 hours ago (timezone slack) to one year ahead. Verified: 30-days-ago → `400` .

P1-11 No session revocation — a stolen token stayed valid for 30 days

A JWT is self-contained, so the only thing that stopped one being honoured was its own expiry. Suspension was enforced per-request (P1-2), but an *individual* leaked token could not be killed at all.

> **Fixed properly:** each account carries a `tokenVersion` , stamped into every token it issues and compared on each request. Bumping it invalidates every token for that account instantly — and it is bumped automatically on password change, deactivation and admin suspension, plus a new `POST /users/me/logout-all` for "sign out everywhere". Tokens issued before this existed count as version 0, so nobody was logged out by the upgrade. Verified by 4 tests, including that a second device dies too and a fresh login still works.

Honest limit: this gives real revocation, not XSS immunity. Tokens still live in `localStorage` , so script injection could still read one — closing that means moving to httpOnly cookies, which changes every client and is a deliberate follow-up rather than something to slip in now.

P1-12 Any cleaner could push a notification to any society

Rate limiting capped the volume but not the authority — a verified cleaner could name *any* society and notify every customer living there. There were also two routes into it, so scoping one would have left the other open.

> **Fixed:** one shared guard, used by both `/agent/notify-area` and `/area-alerts/trigger` . A cleaner may alert only their own assigned area, a society they are scheduled in today, or one where they are currently *enroute/arrived/in progress* — presence beats the date field, so an early arrival or an overrunning job still works. Admins are unrestricted.

P1-13 Referral bonuses were farmable with more phone numbers

> **Fixed:** signups now record a stable per-install device id (sent by the client) alongside the request IP. Past 3 referred signups from the same origin in 30 days the welcome coins are withheld and the account is flagged; the referrer's later payout checks the same flag, so a farmed signup pays out at neither end. Self-referral with your own code is rejected outright.

Deliberately not a block on signup. Indian mobile carriers NAT thousands of real customers behind one address, and families share WiFi — refusing accounts on a shared IP would lock out genuine users. The account is always created and fully usable; only the *bonus* is withheld. That makes farming unprofitable rather than impossible, which is the right trade here.

P1-14 The leads CSV export was executable in Excel

Checking where profile text ends up outside React turned up a real one. `POST /leads` is **public and unauthenticated** with free-form name, city and location. The admin export quoted those fields but quoting does not stop formula injection: Excel and Sheets execute any cell whose text starts with `=`, `+`, `-`, `@`, tab or CR. A stranger could submit a "lead" named `=cmd|'/c calc'!A0` and have it run on an admin's machine the moment they exported and opened the file.

> **Fixed:** a `csvCell()` helper prefixes any dangerous leading character with a single quote so the spreadsheet treats it as literal text. Verified: the four payload shapes are escaped, `Rohit Kumar` and `जॉन कुमार` pass through untouched.

The PDF receipt path was checked too — PDFs don't evaluate formulas, so there is no equivalent there. SMS and push bodies interpolate the name as plain text, which is not injectable, though it is worth knowing a name appears verbatim in outbound messages.

P1-15 `trust proxy` was unset behind nginx

Production serves through nginx, but Express was never told, so `req.ip` was always nginx's own address. Every request on the internet looked like a single client — which silently defeats the IP-based rate limiting added in P1-1 (all admins sharing one bucket) and would have made the new referral IP signal worthless.

> **Fixed:** `app.set("trust proxy", 1)` — exactly one hop. `true` would have trusted a client-supplied `X-Forwarded-For` and been trivially spoofable.

P1-16 The admin dashboard loaded every booking into memory

Found by the volume test. `GET /admin/stats` ran `Booking.find()` with no filter, hydrating every booking into full Mongoose documents and filtering in JavaScript — on every dashboard load.

> **Measured at ~5k bookings:** 4.3 req/s, p95 **3,198 ms**. It degrades linearly, so the first screen an admin opens was on course to become a timeout.

> **Fixed:** counting moved into the database (aggregate + `countDocuments`, run in parallel), plus a `{ status, jobStatus }` index. Re-measured on a larger dataset: **729.5 req/s, p95 15.4 ms — about 170× the throughput and 200× faster at the 95th percentile**, in constant memory.

P1-17 **The session token was readable by any injected script**

The last open item from the earlier passes. Token *revocation* was solved in P1-11, but the JWT still sat in `localStorage`, so an XSS could read it and walk away with a long-lived credential usable from anywhere.

> **Fixed — migrated to httpOnly cookies.** The token is set as `HttpOnly; SameSite=Lax; Path=/` (plus `Secure` in production) and is now unreadable from JavaScript. One cookie per role, so an admin and a customer can still be signed in side by side, with an `X-Session-Role` header picking the right one — mirroring the route-prefix logic the client already used.

- **CSRF** is the new exposure a cookie creates, since the browser attaches it to everything. `SameSite=Lax` handles it in the browser, and an origin check on every cookie-authenticated write is the second layer. Verified: a write from `https://evil.example.com` carrying a valid cookie is rejected **403**, the same write from the real origin succeeds.
- **Precedence matters, and got it wrong first.** Initially the cookie was preferred over the `Authorization` header — which broke 19 tests and, more importantly, was a genuine flaw: a cookie is *ambient*, so a stale one would silently override an explicitly named credential and authenticate the caller as somebody else. An explicit header now always wins.
- **Logging out reaches the server.** Clearing local state alone would leave a working session behind, so the stores' `clear()` now calls `POST /auth/logout` to expire the cookie.
- **WebSockets** can't read a cookie from JS either, so they connect with `withCredentials` and the socket handshake reads the session cookie server-side.
- **Nothing was logged out by the change.** Bearer tokens are still accepted, so existing sessions, API clients and the test suite carry on unchanged.

> **Verified end to end:** login sets an HttpOnly cookie · a request authenticates by cookie alone · no cookie and no header is rejected 401 · cross-site write blocked 403 · same-site write allowed 200 · logout expires the cookie.

UX **The checkout offered payment methods that don't exist**

The payment step asked the customer to choose between **UPI, Card, Wallet and “Pay after service”**. None of them did anything — the selection was local state that nothing downstream ever read, and NexClean only collects cash on completion. It implied a choice the product doesn't offer, and picking UPI or Card set an expectation the app then broke.

> **Fixed:** the selector is gone. The step is now the discount code, a clear total, and the “Pay to cleaner” action. While doing it I found the total was in a sidebar marked `lg:block` — **desktop only** — so on a phone the payment step showed no total at all. It now has a proper breakdown (service total, discount, coins, amount to pay), shown only when something is actually subtracted so an undiscounted booking isn't just the same number printed twice.

5 · P2 — Hardening (all fixed)

- **An admin with no role silently became super-admin.** The permission check called through when no role was assigned, failing open. Now only the seeded super-admin gets the blanket pass; a role-less admin gets an explicit 403.
- **Name validation rejected non-Latin names.** The old ASCII-only rule accepted `O'Brien` but rejected **José** and **जॉन कुमार** — unusable for an India-market app. Now Unicode-aware, covering every script plus the combining marks Indic vowel signs need. Verified: जॉन कुमार → `200`, “Agent 007” → `400`.

- **Area alerts could be spammed.** Any cleaner could push a notification to every customer in any society they named, with no cooldown and no check that they work there. Now capped at 6/hour.
- **Unbounded vehicles and addresses per account** (60 added freely), each embedded in the user document that is loaded on most requests. Capped at 15 each.
- **Booking slots were unvalidated free text.** `"3:00 AM – 4:00 AM (not a real slot)"` was accepted and stored. This mattered beyond tidiness: the overdue-job sweep parses the `"hh:mm AM/PM"` shape, and an unparseable slot yields NaN times that quietly break it. The slot list is now owned by the server and enforced on all four write paths (customer booking, admin reschedule, subscription schedule and reschedule).
- **The admin image upload accepted any string.** It backs two different admin screens so it can't be scoped to a single permission — instead the payload is now validated: it must be a base64 image data URL of an allowed type (png/jpeg/webp/gif/avif — SVG rejected, since it can carry script) and at most 5 MB. A rejection from Cloudinary now returns a 400 the admin can act on instead of an opaque 500.

Verified safe, and left unchanged: NoSQL operator injection is blocked by Zod; customer tokens are correctly refused on admin routes; booking list and cancel are owner-scoped; coin redemption clamps, rejects negatives, and debits atomically — no double-spend race; pricing is fully server-authoritative; unknown package ids return a clean 400; CORS is a real allow-list; Helmet is enabled; and no secret has ever been committed to git.

6 · Test suite health

Before: 87 API failures; then 314 passed / 32 failed / 1 flaky once the two blocking bugs were fixed. **Now: 385 passed, 0 failed, 0 flaky** — full suite, clean database, 6.0 minutes. That includes 14 new landing-page tests and 12 covering previously-untested endpoints (§7), so coverage went up while failures went to zero.

Of those 32 failures only **5 were real product defects** (P1-3 and P0-7). The other 27 were tests that had drifted behind the UI — the more corrosive number, because a suite that cries wolf is one where a real regression goes unnoticed. All are now updated:

#	ROOT CAUSE	WHAT CHANGED
8	Profile UI refactor	Both profile pages now use one shared component ("Save Profile Changes" + inline success, no "Saved" state, no phone-change sub-flow). Specs rewritten against the real form — including a new guard that a Devanagari name saves.
7	Admin selector drift	Reports needed exact matching (new empty-state copy made the old locator ambiguous); payments renamed its heading and tiles; referral settings now sit behind an "Edit settings" toggle; locations addressed by placeholder; leads targets the delete control by title.
6	Name rule vs. test helpers	Helpers built names from `Date.now()`, which the letters-only rule rejects. Added a `uniqueName()` helper that encodes the same entropy as letters. (Mind the prefixes too — "Search **E2E** Customer" fails on the digit.)
4	Subscription checkout label	Now *"Pay to cleaner ₹…"*.
2	Booking flow	The happy path hardcoded the 09:30 AM slot on **today**, so it passed in the morning and failed every afternoon once the app correctly hid elapsed slots. It now books tomorrow and takes whichever slot is offered.

Two genuine UI gaps surfaced while doing this, and were fixed in the app rather than papered over in the test: the payments table had **no empty state** (it just rendered an empty table body), and the name field **stripped non-Latin characters as you typed** — which would have blocked Devanagari input even after the server-side rule was fixed.

Three flaky tests and one case of cross-test pollution were also tracked down and fixed. Worth calling out because an intermittently failing suite trains people to re-run rather than investigate, which is how a real regression slips through:

- **Two agent-signup tests** failed perhaps one run in three, with the page still on the details step and no error shown. The step only advances once the OTP send resolves, so asserting the next heading on Playwright's default 5-second timeout was racing the request. They now wait on the actual `send-signup-otp` response — 7/7 across three consecutive runs with retries disabled.
- **Cross-test pollution** (self-inflicted, from my own rewrite): the agent-profile test renames the shared seeded cleaner to "Rohit Kumar QA" and restores it, but the success banner lingers ~3s, so the restore's assertion passed instantly against the *previous* save's banner — the page then tore down mid-request, leaving the cleaner renamed and failing the admin agent-detail spec that expects "Rohit Kumar". The save now waits for the PATCH itself, and the restore runs in a `finally` so it happens even if an assertion fails.
- **A vehicle-brands test** asserted a new model chip immediately after the POST, racing the list refetch. It now waits for the refetch.

A side effect worth noting: rewriting `requireRole` for P1-2 also fixed a **long-documented bug** the suite had been carrying as five `[BUG: returns 401]` annotations. A valid token with the wrong role used to return `401 "Invalid or expired token"` because the 403 was thrown inside `requireAuth`'s own try/catch. Verified directly: wrong role now returns **403**, missing token still **401**. The stale annotations are cleared.

7 · Endpoint coverage

To answer “is anything left untested?” properly rather than by feel, I enumerated every route the backend mounts and cross-checked each against the whole test tree. That found **10 of 156 endpoints referenced nowhere** — including several that are destructive or security-relevant, which is exactly the kind of route nobody notices breaking:

WAS UNTESTED	WHY IT MATTERED
`POST /users/me/deactivate`	Account deactivation — destructive, and it must also lock the user out. Now asserted end to end.
`PATCH /users/me/password`	OTP-gated password change. Now covers wrong code, short password, and unauthenticated callers.
`POST /admin/dispatch/auto-assign-all`	Bulk dispatch across every pending booking, with no role test at all.
`POST /agent/notify-area`	Pushes to every customer in a society; now also asserts a customer can't trigger it.
`PATCH /subscriptions/schedule/:id`	Wash reschedule — now asserts free-text slots are refused.
`POST /auth/agent/signup-basic`	Creates the cleaner account; now pins `agentStatus: "unverified"` (the state behind P0-7).
+ 4 more	`verify-otp-code` , `unregister-token` , `available-jobs` , `admin/leads/:id/status` .

All ten now have tests (`tests/api/uncovered-endpoints.spec.ts` , 12 cases). Re-running the same analysis afterwards: **156 of 156 routes referenced, 0 uncovered.**

8 · The probe suites

All of these live in `tests/` so any finding can be re-verified, and re-checked after future changes. Each was run repeatedly against a fresh database and reproduced identically.

FILE	COVERS
`adversarial.mjs`	Auth abuse, payment integrity, authorization, edge cases
`adversarial2.mjs`	Token revocation after suspension, referral farming
`adversarial3.mjs`	Support-ticket IDOR
`adversarial4.mjs`	Injection, date bounds, error handling, unbounded growth
`verify-fixes.mjs`	Replays all 10 security fixes in production + online-payment mode
`concurrency.mjs` *(new)*	**7 race-condition probes.** Parallel coin redemption, two cleaners racing one offer, two claiming one job, repeat cancels, simultaneous first-login, and capped-array bursts. Caught P1-10.
`multiworker.mjs` *(new)*	**Cross-worker real-time.** Runs two API workers on one database, connects a socket to A and fires the event from B. Caught P1-8.
`subscription-scheduler.mjs` *(new)*	**The timed loop.** Confirms a recurring wash is auto-assigned to the nearest cleaner ahead of its slot, and that a coordinate-less one goes to the admin queue. 4/4 passing.
`e2e/landing/` *(new)*	**14 browser tests** for the previously untested public site: all story sections, nav scrolling, hash deep-links, the 404 fallback, phone-viewport layout, and all 8 policy/content pages. Caught P1-9.

Run the probes against a throwaway backend only — never point them at production.

9 · Volume testing, push, and what is genuinely left

Load test

`tests/volume.mjs` drives four representative paths with concurrent clients for a sustained period. Results after the P1-16 fix, 30 concurrent clients:

SCENARIO	THROUGHPUT	P95	ERRORS
Public catalog read — `GET /catalog/packages`	9,897 req/s	4.9 ms	none
Authenticated read — `GET /bookings`	1,785 req/s	21.5 ms	none
Write + dispatch — `POST /bookings`	337 req/s	42 ms	none
Admin aggregate — `GET /admin/stats`	730 req/s	15.4 ms	none

No 5xx and no dropped connections in any scenario. The authenticated read carries the extra per-request account lookup that token revocation needs, and still sustains ~1,800 req/s — that design choice is comfortably affordable. A separate burst of 300 simultaneous bookings from one account returned 300 × 201 with no conflicts.

Push notifications

Now configured and verified live: the Firebase service account initialises against project `nexclean-19d5c` and a real FCM call is made (a deliberately invalid token is rejected by Google, which is the proof that the credentials and request are genuine). The invalid-token cleanup path also runs as intended. Push is no longer in mock mode.

> **Rotate that service-account key**
> The Firebase private key was pasted into a chat transcript to configure this. It is stored only in `backend/.env` (git-ignored, never committed), but it now exists outside your secret store. Generate a replacement in Google Cloud Console → IAM & Admin → Service Accounts → Keys, and delete key id `5d0d8a9e…e964`.

Genuinely still open

ITEM	WHY IT IS STILL OPEN
Razorpay refunds	Blocked at the account, not in code — see P0-9.
Soak / endurance	Sustained bursts are covered; nobody has run the API for hours to surface slow leaks or connection-pool exhaustion.

One environmental note: a few E2E tests assert *zero* console errors, so they fail on any transient network blip (the frontend loads Firebase at runtime). That happened once during this session and is worth knowing before anyone reads such a failure as a regression.

10 · Before you deploy

Everything fixable in code is fixed and the suite is green. What is left is yours:

1. **Razorpay refunds (P0-9)** — the one genuine blocker. You do *not* need the Payouts API; refunds run on the Payments API your key already has. Try the refund from the Razorpay Dashboard on that payment: it will state the real reason, most likely no balance to draw from (₹2,979 of ₹3,050 is already settled out). Until it works, no customer can be paid back.
2. **Set `REDIS_URL`** — noted as yours to add. Without it, live chat and job offers are silently dropped for anyone whose socket sits on a different PM2 worker. Verify with `node tests/multiworker.mjs`.

3. **Rotate the Firebase service-account key** — it was pasted into chat to configure push. Stored only in the git-ignored `.env`, but replace key `5d0d8a9e…e964` anyway.
4. **Confirm the server `.env`** — you've said `NODE_ENV=production` is already set there. The new startup guard will refuse to boot until `DEMO_OTP` is empty, `ADMIN_PASSWORD` is changed from `password123` and `JWT_SECRET` is a real secret, so a bad config fails loudly instead of silently.

Two deliberate non-changes: `PAYMENTS_ONLINE` stays `false`, matching the pay-to-cleaner flow the app actually ships (the payment-integrity fixes are what make turning it on safe); and rate-limit counters remain in memory, so they are per-worker — move them to Redis when you add it if you want an exact global ceiling.
